Empirical Insights into Chat Generative Pre-trained Transformer-Assisted Code Refactoring and Developer Dynamics

James Grant[a,1], Jocelyn Mo[a,1], Bowen Jiang[a,1], Eman Abdullah AlOmar[a,*]

[a]*Stevens Institute of Technology, Hoboken, NJ, USA*

Abstract

Large Language Models (LLMs) have become an increasingly popular tool in all aspects of code development. By now, the capabilities of LLMs in code production have been substantially researched. Despite recent studies investigating the capabilities of LLMs in code production, little is known about the practical use of LLMs by developers in a refactoring context. In an effort to close this gap, we conducted an empirical study based on interactions between developers and Chat Generative Pre-trained Transformer (ChatGPT). Using Developer-Generative Pre-trained Transformer (DevGPT) dataset, our study aimed to further preliminary research on how developers can use ChatGPT to refactor code effectively. To do so, we focus on the extent to which ChatGPT was helpful, the prompt that gives a successful answer in the fewest interactions, and the programming languages in which ChatGPT is most effective. We found that, overall, ChatGPT provided developers with code that they could directly utilize in their projects more often than code that needed modifications. Our prompt taxonomy suggested that developers can often avoid lengthier conversations by using more direct prompts that define ChatGPT's role or providing code snippets, rather than the approach of requesting code diagnosis, explanations, or code generation. Finally, ChatGPT demonstrates varying proficiency levels in different programming languages, with Cascading Style Sheets (CSS) receiving the most effective support. These insights provide valuable guidance for developers looking to optimize their use of ChatGPT for code refactoring.

Keywords: Refactoring, Software Quality, Software Engineering, Large Language Models, Artificial Intelligence

1. Introduction

*Corresponding author

Email addresses: jgrant4@stevens.edu (James Grant), jmo6@stevens.edu (Jocelyn Mo), bjiang14@stevens.edu (Bowen Jiang), ealomar@stevens.edu (Eman Abdullah AlOmar)

[1]Authors contributed equally

1.1. Motivation

Ever since the popularization of Large Language Models (LLMs) after the release of OpenAI's Chat Generative Pre-trained Transformer (ChatGPT-3) in 2022, they have been increasingly utilized as a tool for developers Hao et al. (2024); Chavan et al. (2024); Ma et al. (2023); Biswas (2023); Dilhara et al. (2024); Shirafuji et al. (2023); White et al. (2023, 2024); Sun et al. (2023); Tian et al. (2023); Liu et al. (2023); Haque & Li (2023); Sobania et al. (2023); Xia & Zhang (2023); Khoury et al. (2023); Tufano et al. (2024); Liu et al. (2024); AlOmar et al. (2024); Yokoi et al. (2023); De Martino et al. (2024); Xiao et al. (2024a); AlOmar et al. (2025); Divyansh et al. (2025). It is crucial to investigate how they can be used most effectively and in what circumstances developers should avoid using the aid of LLMs. Particularly, there is a need to tackle the trial-and-error process in AI-assisted refactoring. While LLMs can generate code, refining existing code demands high precision to preserve functionality. Developers currently struggle with prompt engineering, often spending more time "negotiating" with the LLM through multiple attempts than refactoring manually. It is important to identify prompts that produce "successful responses with the fewest interactions", thereby offering a direct way to reduce developer burnout in AI-driven code maintenance Martinez et al. (2025).

1.2. Literature Review

Several recent studies AlOmar et al. (2024) AlOmar et al. (2025) Hao et al. (2024) Jin et al. (2024) Xiao et al. (2024b) Yu et al. (2024) have began to analyze the behavior of LLMs, including ChatGPT, to test to see if it is a reliable tool in the development process. To simulate how a developer might posit their struggles through the coding process, Yu et al. have manufactured different "prompts" which are various ways that developers are likely to question ChatGPT Yu et al. (2024). These prompts include asking ChatGPT a question guided to a resolved code, or simply printing a test report about the issue. While previous studies suggested that methods other than directly asking the LLM will lead to more accurate results, for example, asking a "guiding question" to ChatGPT will lead to up to a 59% increase in successfully identifying failed repairs within code, a common conclusion is that ChatGPT-3 is an unreliable model for both generating accurate code and identifying the issues in buggy code Yu et al. (2024). Other studies have begun to focus on analyzing actual conversations between programmers and LLMs using datasets of these conversations provided using open source repositories Xiao et al. (2024b) Jin et al. (2024) AlOmar et al. (2024). These datasets can include tens of thousands of messages sent between the user and an LLM, sometimes resulting in a snippet of code that a programmer found helpful or even copied exactly to use within their code. Jin et al. have parsed through these conversations with the goal of analyzing the accuracy of ChatGPT's code generation and its reliability to create a snippet of code simply based off a description from the user Jin et al. (2024).

1.3. Objective

Although studies were previously mentioned Yu et al. (2024) Jin et al. (2024) have focused on the lack of ability of LLMs to generate code and to detect errors in its generated code, research on the accuracy of refactored code implies that LLMs excel at generating refactoring suggestions, yet not all of these suggestions are useful Dilhara et al. (2024); Pomian et al. (2024a). If developers cannot create the ideal code that they envision for their program simply based on a description, the ability to have ChatGPT read already formatted code and form it into the most ideal version could make this a reality as well, thus causing a need to analyze ChatGPT's refactoring ability. The potential of ChatGPT to be used as a tool to refactor already produced code is substantial, but it is difficult to predict how accurate its responses can be in different programming contexts Hao et al. (2024). Where previous studies have focused on the accuracy of refactored ChatGPT code or how developers have utilized ChatGPT, our study aims to clarify how ChatGPT in its current state is best able to aid developers and focuses on what ChatGPT specializes in during the software refactoring process.

1.4. Results

Our results indicate that ChatGPT can have a positive impact on the refactoring process, showing that 97% of the refactored code given by ChatGPT in the 'commit' category were merged to the final repository exactly as they were given in the conversation. Developers providing context or role designation ("Act as..") in their prompts seemed to be the best route, as these prompt categories yielded the lowest average number of turns before the developer was satisfied with a response. Finally, overall, Cascading Style Sheets (CSS) was the most common language utilized by developers in the dataset, and also the language with the highest percentage of merged instances. Typescript was a popular language in the 'Pull Request' and 'Issue' categories, although it held a low merge rate, suggesting that developers working on large, complex projects may be unsatisfied with ChatGPT's refactoring suggestions.

1.5. Contribution of the Article

The contributions of this paper are summarized as follows:

- Our study offers valuable insights into how developers use ChatGPT for code refactoring. Through examining a dataset of prompts and responses, we identify patterns and trends that highlight both the strengths and difficulties of the tool.
- This paper not only assesses ChatGPT but also offers actionable advice for developers on improving collaboration with AI models in code refactoring. These suggestions are based on our analysis of developer-ChatGPT interactions.
- Our experiment is publicly available for replication and extension Rep.

2. Related Work

Pomian et al. Pomian et al. (2024a,b) investigated the integration of Large Language Models (LLMs) with Integrated Development Environment (IDE) static analysis approaches. They introduced Extract Method Assistance (EM-Assist), a tool that runs as an IntelliJ IDEA plugin and leverages static analysis and the creative powers of LLMs to propose and implement Extract Method refactorings that developers find most convenient. AlOmar et al. AlOmar et al. (2024) investigated how developers and Chat Generative Pre-trained Transformer (ChatGPT) communicate when it comes to code refactoring. The results show that generic and particular refactoring phrases are regularly used in developer-ChatGPT conversations, with developers typically making general requests and ChatGPT responding with precise refactoring intentions. Prompt-based LLMs have recently been released by OpenAI and have shown impressive features and responses based on its advanced learning capability Yu et al. (2024). In their study, Yu et al. stated that they have observed previous studies that praise the model for its effectiveness in software development, namely code generation, code completion, and program repair. Jin et al. suggested that developers are likely to take an interest in LLMs for their use in software development, particularly after previous studies have shown reliable code generation Jin et al. (2024). Hao et al. Hao et al. (2024) conducted a study focusing on the nature of how programmers engage with ChatGPT by analyzing the prompts in shared conversations. The authors developed a 16-category taxonomy for the initial prompts, providing a potential guide on how to further develop FM, and analyzed the flow patterns. DePalma et al. analyzed the refactoring ability of ChatGPT after noticing a limitation of the software's code comprehension and interpretation abilities DePalma et al. (2024). The study studies the effectiveness of ChatGPT's refactoring, the functionality of its refactored code, and how insightful ChatGPT's generated documentation can be. Deo et al. sought to identify how ChatGPT is used by developers through the refactoring process to broadcast how useful ChatGPT can be in the software development process Chavan et al. (2024). The authors found that most of the conversations with ChatGPT involved either a new feature being worked on in the program, or an error within the code. White et al. White et al. (2024, 2023) presented a collection of prompt patterns in order to deal with common software engineering challenges. The study highlighted how LLMs, can help automate software development processes more successfully, but it also notes that human monitoring is still necessary because of the models' propensity to provide inaccurate outputs. To mitigate unnecessarily complex programs, Shirafuji et al. Shirafuji et al. (2023) developed and tested a few-shot prompting methodology that used ChatGPT to refactor users' Python programs to increase readability and maintainability while maintaining the correctness of the program. These studies have resulted in an improvement of our understanding of the potential of ChatGPT's assistance as a generative software testing and development tool. Rather than examining ChatGPT's coding accuracy, our methodology grounds its analysis in developer conversations and refactoring tasks to gain an understanding of how

developers can best prompt the LLM to output a successful refactored program.

The current automated refactoring methods use machine learning, static analysis, and metric-based approaches that operate within specific task boundaries and controlled environments. Research on IDE plug-ins that integrate LLMs with static analysis focuses on specific refactorings, such as Extract Method, but requires complete tool integration. The signal-processing-based techniques convert code metrics and execution traces into numeric signals, which then get filtered or thresholded to detect abnormal patterns. The methods identify potential problems through their analysis, can be sensitive to noise and workload changes. Also, they require manual threshold adjustment and produce general alerts that do not help developers make concrete, context-aware refactoring suggestions.The research evaluates ChatGPT using predefined refactoring tests, in which researchers establish both the input prompts and the evaluation standards. The techniques offer specific transformation guarantees, but they fail to observe how developers handle suggestion negotiations, whether generated code becomes part of the final product and how different prompt formats affect the interaction process. The DevGPT-based approach extends previous research and has the advantage of monitoring actual developer-ChatGPT dialogues, which connect to GitHub commit records to measure merge success rates, identify successful prompt formats, and evaluate refactoring performance across different programming languages. The main limitations of our method are that it relies on a single, self-selected dataset, some conversation categories and languages are underrepresented, and we cannot see refactorings that developers performed with ChatGPT but chose not to share. The research sacrifices experimental control and broad scope to achieve a better understanding of LLM-based refactoring in real-world development environments.

3. Study Design

The main goal of this study is to recognize the current programming capabilities of language models such as Chat Generative Pre-trained Transformer (ChatGPT), and analyze what steps developers can take to utilize them as tools to help refactor or repair their current projects. Where earlier methods have attempted to simulate how a developer would utilize ChatGPT, our proposed method improves accuracy by analyzing real-world conversations and refactoring requests from developers. Figure 1 summarizes our study design, starting with the Developer-Generative Pre-trained Transformer (DevGPT) dataset, which is a set of instances with GitHub content linked to ChatGPT conversations. This visual flowchart clarifies how each RQ builds on a clearly defined subset of the data, strengthening the transparency and interpretability of our study design. These are real users who shared ChatGPT interactions for open-source GitHub projects, which allows us to analyze authentic user behavior.

3.1. Developer-Generative Pre-trained Transformer (DevGPT) Dataset

Our research was run on the DevGPT dataset, which is an amalgamation of 29,778 prompts and responses sourced from open source GitHub repositories,

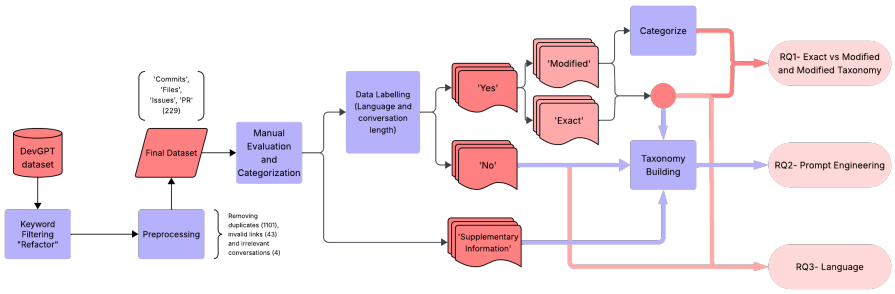


Figure 1: Visual representation of our overall approach to utilizing the dataset to best research our areas of interest.

including links to source code, 'Commit', 'Issue', 'Pull Request', 'Discussion', and 'Hacker News' Threads Xiao et al. (2024b). This dataset has been used in several software engineering tasks, including refactoring AlOmar et al. (2025, 2024); Chavan et al. (2024), code review Chouchen et al. (2024), and bugfix Mondal et al. (2024); Das et al. (2024). We filtered this database to select only the instances that contained the "refactor" keyword in the project, including the ChatGPT prompt, response, issue title, etc. The further breakdown of instances into 'Commit', 'Issue', 'File', 'Pull Request', 'Discussion', and 'Hacker News' can be found in Table 1, which shows the dataset composition and filtering steps, highlighting the rarity of explicit refactoring conversations. We began processing this subset by removing duplicate instances, invalid links, and irrelevant conversations. Table 1 additionally shows that after filtering the dataset for the "refactor" keyword, only 1,389 instances remain, meaning that less than 5% of the original dataset includes developers discussing their refactoring process. After filtering for refactoring and removing duplicates and invalid links is complete, we yield 229 unique analyzable instances across Commit, File, Issue, and Pull Request categories. Each instance was pasted into a spreadsheet including each instance's category type (Commit, File, Issue, Pull Request, Discussion, Hacker News), the name of the author of the repository, the language of the repository, a url linking to the repository on GitHub, and the share link for the corresponding ChatGPT conversation.

3.2. Data Analysis

The first round of processing was set with the goal of manually analyzing the instances, including recording conversation length and the extent to which the refactored code was merged to the project. Since the spreadsheet was organized by category type, each of the 3 authors evenly split each sheet, named as one of the 6 category types (Commit, Issue, File, Pull Requests, Discussion, and Hacker News). It became clear that multitudes of instances were duplicate instances, had invalid links, or were linked to irrelevant conversations. We disregarded all of these instances accordingly. For the 'Discussion' and 'Hacker News' categories, this left too insignificant of a number. Thus, we disregarded 'Discussion' and 'Hacker News' as well. This first round of processing filtered a

total of 1,101 duplicate conversations from the dataset, as well as 47 invalid or irrelevant shared ChatGPT links, narrowing down our dataset to 229 instances, as seen in Table 1. It is important to note that a small portion of these instances were found to have conversations with no code generated, but included content directly related to refactoring tasks and aided the programmer in some form. These instances were named 'Supplementary Info.'

Next, to explore our 3 points of interest, the data we needed to collect for each instance was as follows: 1) whether or not the refactored code was merged to the final repository ('Yes' or 'No'), and if so, whether the exact code snippet or a modified version was used ('Exact' or 'Modified'); 2) the length of the ChatGPT conversation, defined as the amount of messages sent by the Anonymous user; and 3) the programming language the project was written in. While the language was already provided in the spreadsheet, the data needed to be reviewed and sometimes revised. It is important to note that 1) and 3) were recorded for instances not marked 'Supplementary Info.' Again, the instances in each category type were evenly divided between the authors for review, which is approximately 1/3 of the data for each author. If any instance was deemed difficult to assign to 'Exact', 'Modified', or 'No', the authors highlighted the instance and discussed whether the code snippet fit into each category. Any disagreements of how the data was labeled that were found upon review by the authors were discussed at length until an agreement was reached. Disagreements among authors included discussing whether the modifications on a code snippet were too heavy to be included in the 'Modified' category, or if one author struggled to see where a code snippet was implemented within the GitHub repository. All instances were assigned to one of the three aforementioned labels, and all authors were in agreement regarding issues with labeling after each instance was discussed.

Finally, the data points were reassigned to each author according to category type. One author was assigned to 'Commit,' one to 'File,' and one to 'Pull Request' and 'Issue.' In addition to being another level of data review, this allowed the authors to focus on the data in their respective categories, familiarizing the authors with a designated subset to set up further analysis and aggregation of information into textual and visual representations. The specifics for this further analysis in each RQ are outlined below.

The application method of our approach differs from previous refactoring techniques because it uses different validation procedures. Prior research on machine-learning and static-analysis-based refactoring tools, including IDE plug-ins that link LLMs to Extract Method refactorings, few-shot prompting methods for enhancing code readability and maintainability, focuses on controlled tasks and curated benchmarks to measure success through static metrics, test-suite correctness, and code-quality scores. In contrast, our method operates directly on real developer–ChatGPT conversations from the DevGPT dataset (see Section 3.1) and validates its findings using features that reflect actual development outcomes. As detailed in Section 3.2, for each instance we record whether the ChatGPT-generated code was merged to the repository and in what form ("Exact", "Modified", or "No"), the length of the developer's side of the conversation,

Table 1: Summary of the extracted data

Item	Count
Shared ChatGPT links	4,733
GitHub or Hacker News references	3,559
ChatGPT prompts and response	29,778
Code snippets	19,106
Instances having keyword '*refactor**'	1,389
Refactoring GitHub commits	703
Refactoring GitHub issues	143
Refactoring GitHub code files	398
Refactoring GitHub pull requests	133
Refactoring GitHub discussion	9
Refactoring Hacker News	3
Duplicate instances	1101
ChatGPT link is not working	43
Irrelevant ChatGPT link	4
Final dataset	229

and the project's primary programming language and artifact type (Commit, File, Issue, Pull Request). On top of that, the research develops a four-way taxonomy of modification types (see Table 2) and prompt categories (see Section 3.4), which enable us to link fundamental features to the results presented in RQ1–RQ3. The feature set enables us to validate raw conversations through transparent statistical aggregation, which provides an alternative view to previous studies. The evaluation method tests LLM-assisted refactoring through actual merge decisions and developer effort in different project environments instead of using syntactic correctness and code-quality scores.

3.3. RQ1: How helpful is the code generated by ChatGPT in assisting developers with refactoring tasks?

RQ1 aims to explore to what extent the generated code is used. As explained in Section 3.2, we manually analyzed each instance of the DevGPT dataset, and compared the code provided by ChatGPT to the corresponding GitHub link. Each instance was categorized as exactly merged, modified, or not merged at all. We considered only 'Exact' and 'Modified' instances for RQ1. Regarding the 'Modified' category of instances, not all modified code resulting from conversations with ChatGPT were altered in the same way, but were grouped together within the 'Modified' category to simplify the analysis. Table 2 displays the 4 methods developers typically used to modify the code received from ChatGPT during their conversation, these frequencies illustrate the spectrum of developer engagement and validate our observation that most modifications involve modest, targeted edits rather than extensive rewrites. The categories within this taxonomy were agreed upon after every author inspected each of the 28 Modified instances to note how the resulting code from ChatGPT was

utilized and convened to discuss the commonalities of all instances. After each instance was discussed, the authors created the 4 categories displayed in Table 2 to summarize how the developers in the dataset were able to implement their modified code. These categories were created after analyzing the notes made for each instance and merging similar notes into a singular category, resulting in 4 unique categories. If the code from the conversation was modified in any of these 4 ways in the given repository, it was considered a 'Modified' instance.

3.4. RQ2: What is the best way to prompt ChatGPT to get a satisfactory response about refactoring with the fewest possible interactions?

RQ2 focuses on what kind of prompt given to ChatGPT translates to successful answers in the fewest possible ChatGPT interactions. We used the conversation lengths (see Section 3.2) and the type of prompt the developer opened the conversation with. The prompt engineering process is as follows: to attempt to find the best prompt that elicits the fewest possible interactions, we decided to place each instance into prompt categories. Each author manually analyzed the ChatGPT conversations in their respective category types, focusing specifically on the initial turn, unless the refactoring task did not appear until after the initial turn. We noted any distinct phrases seen in multiple instances, code snippets, and common keywords. The authors then used these notes from each instance to produce potential prompt categories for their own category type such as, begins with "From now on act as...". We compared our prompt groups and decided which were similar, necessary, or unnecessary. Prompts that only concerned a specific instance were deemed unnecessary and were accommodated into a broader prompt category. Prompts that were specific and correctly assessed at least 5 instances were deemed necessary to keep. We merged them into 7 final prompt categories, considering all instances and making sure each comfortably fit into a prompt category. All instances were divided into these groups by their respective author, and the average number of turns was calculated for each prompt. The 'No' and 'Supplementary Info' instances were considered separately.

3.5. RQ3: With which programming language does ChatGPT give the most satisfactory responses about refactoring?

RQ3 aims to find which programming languages ChatGPT is statistically most effective with. For each subsection, 'Commit,' 'File,' 'Issue,' and 'Pull Request,' we grouped instances by programming language, analyzing what percentage of each language group were merged instances, and whether they were exact or modified.

4. Experimental Results

4.1. RQ1:How helpful is the code generated by ChatGPT in assisting developers with refactoring tasks?

Motivation. It is critical to determine the usefulness of Chat Generative Pretrained Transformer (ChatGPT) generated code in aiding developers with both

writing and refactoring codes. The efficient generation of code and refactoring enhance the quality of codes by improving their maintainability and readability, reducing bugs that arise during future development; the task is resource-intensive and time-consuming. Hence, suggestions by ChatGPT have the potential to save a significant amount of time plus effort for developers which they can allocate to more complex tasks. Moreover, ChatGPT can be a resource for junior developers to teach them how best practices are done— which in turn helps them to level up their coding skills. In addition, with ChatGPT aiding in quick prototyping without needing physical components, developers have an opportunity to try different design patterns or structures error-free. It is from this perspective that investigating the impact of ChatGPT on software engineering productivity through its effectiveness in writing and refactoring code quality can lead an organization towards cost reduction with higher quality products developed within shorter timelines.

Approach. To develop the taxonomy of the modified code instances, a detailed manual analysis of the dataset was done, where authors tried to identify common themes and patterns in how developers modified ChatGPT-generated code. The modifications were categorized into four distinct groups: 'Variable Name Change,' 'Used Structure, 'Changes Function to Fit Specific Project,' 'Asked for General Advice and Altered Code to Fit the Program,' and 'Provided the Exact Code and Asked for Tips to Improve It.' The first category, 'Variable Name Change,' comprised cases where developers had changed variable names only for uniformity within their codebase. Developers retained the overall structure of the generated code but modified specific functions to better suit their project requirements in the 'Used Structure and Changes Function to Fit Specific Projects' category. Developers seeking general advice from ChatGPT and then adapting the suggested code to fit their specific needs were involved in the 'Asked for General Advice and Altered Code to Fit the Program' category. The 'Provided Exact Code and Asked for Tips to Improve It' category included instances where developers requested tips to enhance the exact code provided by ChatGPT and incorporated the feedback into their projects.

The authors used unique instances found in the DevGPT dataset that were found to be merged into the final code found in each repository, all of which we parsed out from the dataset to answer this question. After three authors used URLs to 1389 instances on GitHub and then examined them, inspection and improvement were conducted during the iterative labeling process to guarantee the labels' correctness and clarity. After discussions to address the conflict, a final list of 229 classified occurrences was produced. 12.22% of instances are modified and four categories were categorized that can best represent the reason why the instances were modified, and taxonomy was created to help answer the question, see Table 2.

Results. For each modified instance, four types of reasons for being modified are summarized in order to create taxonomy. The first category is asking ChatGPT to improve the code, 37.18% of the developers will throw the whole code into ChatGPT and ask ChatGPT for improvement, and 23.14% of the time developers are not satisfied with the result so changes were made. The

Table 2: Taxonomy of Modified Count/Frequency

	Category	Count
I	Provided the exact code and asked for tips to improve it	1
II	Asked for general advice and altered code to fit the program	15
III	Used structure, changes function to fit specific project	6
IV	Variable name change	6

second category is asking ChatGPT a vague question, which will cause ChatGPT to answer the wrong part of the question or not provide the answer the developer wants which is very common. The third category is changing structure, in which the developer would ask ChatGPT to create a different format of code/read me to what the developer already has and the developer would end up making minor changes to what ChatGPT provided. The last category is a variable name change, which is the developer asking ChatGPT to change the variable and fix the code accordingly. Creating a taxonomy helps to better analyze the reason and gather statistics.

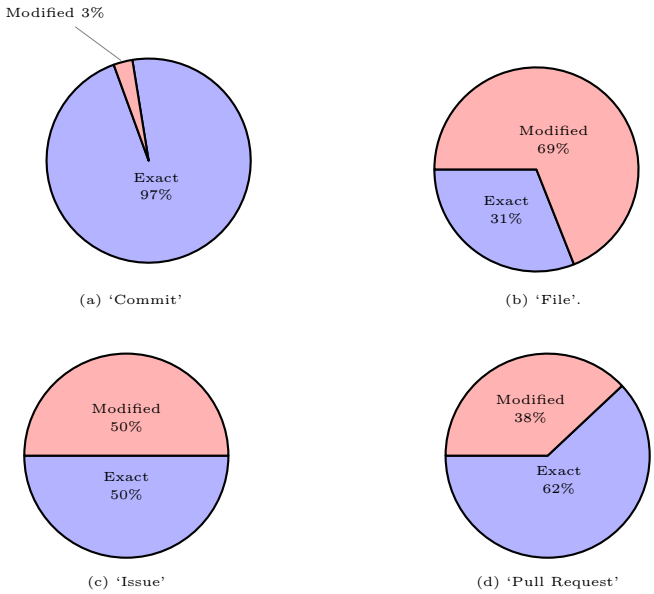
'Exact' instances are defined as when the developer uses the code provided by ChatGPT line by line without any modification or deletion. On the other hand, 'Modified' instances are defined as when the developer makes changes to the code including but not limited to optimization, adaptations, and bug fixes. The 'No' instance in this study means the developer did not adopt ChatGPT's code at all. After the data was gathered, a total of 63.32% of instances were found to be exact, a total of 12.22% of instances had been modified, a total of 10.91% of no instances were found, and the rest being 'Supplementary Info' cases with no code.

4.1.1. Commits

A total of 138 Commits instances were found in the dataset where code generated by ChatGPT was integrated. These were split into exact versions and modified versions. The predominance of the former over the latter indicates that using ChatGPT is helpful and effective for developers during refactoring works over Commits. In most cases, it demonstrates the tool's promise as an asset for development, despite some minor modifications that may be needed after insertion.

4.1.2. Files

In the 'File' category, there were 56 distinct instances after removing duplicates and invalid links. Out of the 31 instances where the ChatGPT conversation produced code, 9 utilized the code exactly as generated, 20 utilized a modified version of the code, and 2 did not merge the provided code at all. Despite the need for adjustments in around two-thirds of the instances, the code was generally able to be integrated into the project in some form. Figure 2 displays the total number of instances that were merged to the repository, as well as a breakdown of the number of 'Exact' and 'Modified' instances per category. As shown in Figure 2, the 'Commit' category had the largest number of 'Exact' instances, while the 'File' category had the largest number of 'Modified' instances.



(a) 'Commit' (b) 'File'.

(c) 'Issue' (d) 'Pull Request'

Figure 2: The frequency of Exact vs. Modified instances

4.1.3. Issues

The amount of instances in the 'issue' category that contained code snippets that were pushed to the final version of the repository was minuscule compared to other categories. Out of 22 individual instances, only 2 were found to be merged, one being exact and one being modified. While some of the instances were not able to be analyzed due to an improper ChatGPT link provided within the GitHub repository, a majority of the cases simply did not use the code that was provided by ChatGPT after asking the program to aid in resolving the issue.

4.1.4. Pull Request

The number of total unique pull request instances within the dataset is 22, 8 of which contain code that was generated by ChatGPT and merged to the final version. 5 of these 8 fit into the Exact category, leaving 3 to fall into the Modified category as they contained slightly altered versions of the code given by ChatGPT. While the number of requests containing any version of ChatGPT generated code that was merged to the repository is small in comparison to the 'Commit' category, the data here show that developers found ChatGPT to be a somewhat useful tool during the review or refactoring process. of their code

RQ1 Summary. Through the analysis of DevGPT dataset instances, we investigated the degree to which ChatGPT-generated code aids developers in refactoring tasks. The outcomes showed that ChatGPT can produce code that developers can use immediately or with little to no modification. For instance, a sizable portion of the cases in the 'Commit' category featured developers use the precise code that ChatGPT supplied. This shows that ChatGPT can produce useful refactoring recommendations, saving developers time and effort. However, developers frequently needed to make additional changes to the generated code for more complicated tasks or less popular programming languages. This shows that while ChatGPT is a useful tool for simple refactoring jobs, the more particular and sophisticated the work, the less effective it is; in these cases, human monitoring and knowledge are essential.

4.2. RQ2: What is the best way to prompt ChatGPT to get a satisfactory response about refactoring with the fewest possible interactions?

Motivation. The analysis of the ChatGPT conversations revealed that programmers had varying approaches to prompting ChatGPT for an answer. The objective of RQ2 was to categorize these prompts in order to study what prompt style was the most effective. Using the preliminary data found researching RQ1, we revisited all merged instances and put them into our prompt taxonomy. For each prompt category, we calculated the average number of turns. We aimed to deduce which prompts had the lowest average number of turns, significantly focusing on the efficiency of ChatGPT conversations, rather than just to what extent they were effective.

Approach. For the first part of RQ2, we manually processed the conversation length by counting the number of turns sent by the user (see Section 3). Including the 'Yes' and 'Supplementary Info' instances only, we calculated the average number of turns for each category type, 'Commit,' 'File,' 'Issue,' and 'Pull Request.' In order to explore the prompt that elicits a successful answer in the fewest possible interactions, we decided to place each instance into prompt categories. The prompt engineering process is explained in detail in Section 3.4. We ended up with 7 final prompt categories, and all instances were divided into these groups by their respective researcher. The average number of turns was calculated for each prompt, including only the 'Yes' instances. We considered the 'No' and 'Supplementary Info' conversations separately.

Results. Figure 3 displays the box plot representation of the distribution of conversation lengths for each category type. The mean number of turns for 'File' is drastically greater than the means for the 3 other categories. The lengths of 'File' instances also range from 1 to 54, not even including outliers with lengths greater than that. The reasons for this drastic difference are discussed in Section 5. 'Pull Request' and 'Commit', however, have strikingly similar means and the same interquartile range. Both have a couple outliers exceeding this range. In the following section, we describe each prompt category in more detail. (I) The

'From now on act as...' category includes instances that initialize the conversation with this phrase, telling ChatGPT to impersonate a professional in order to carry out the following task. (II) 'Provide direct code' instances initially prompt ChatGPT with code snippets or Files, possibly accompanied by supplementary description sentences. (III) Similar to Category I, the 'You are... then provide direct code' prompt begins by telling ChatGPT that they are to impersonate an expert before explaining the task and providing code snippets. (IV) 'Give code and ask to restructure' instances prompt ChatGPT with a code snippet and request to refactor or optimize specific functions. (V) The 'Give code and ask to diagnose the problem' category also begins with introduction of code and follows with a request to review the code and find the source of the issue. (VI) The 'General question before task' prompt category includes the instances that initialize the conversation with a general question and eventually ask specifying questions about their own code or the code produced earlier in the conversation. (VII) The last prompt, 'Generate code and refactor,' is found only in the 'Files' category. It refers to conversations started by a request to write code for an app or to implement a function. The programmers later prompt ChatGPT to refactor the code initially produced.

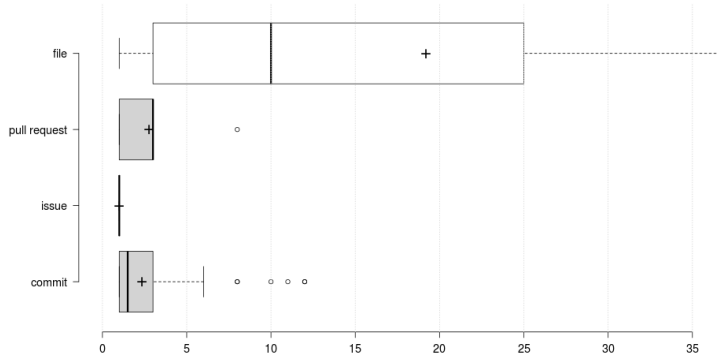


Figure 3: Distribution of conversation lengths for 'Commits,' 'Files,' 'Issue,' and 'Pull Request'

4.2.1. Commits

According to Table 3, giving ChatGPT a direct piece of code to work with or telling it to take on a specific role or behavior using the 'From now on act as...' prompt is the most efficient way to get a satisfactory response about refactoring with the fewest interactions. These two kinds of prompts are the most commonly employed suggesting that they are successful in producing acceptable outcomes with the least amount of involvement.

4.2.2. Files

Table 4 depicts the prompts across the instances in the 'File' category. Most instances belong to the 'Provide direct code' or 'Generate code and refactor'

Table 3: Prompt categories for 'Commit' and their average turns

	Prompt	**Avg Turns**
I	From now on act as...	1.58
II	Provide direct code	2.06
III	You are... then provide direct code	3.57
IV	Give code and ask to restructure	4.36
V	Give code and ask to diagnose the problem	10.33

Table 4: Prompt categories for 'File' and their average turns

	Prompt	**Avg Turns**
II	Provide direct code	24.4
III	You are... then provide direct code	5
IV	Give code and ask to restructure	3
V	Give code and ask to diagnose the problem	1
VI	General question before task	26.2
VII	Generate code and refactor	26.18

Table 5: Prompt categories for 'Issue' and their average turns

	Prompt	**Avg Turns**
IV	Give code and ask to restructure	1
VI	General question before task	1

Table 6: Prompt categories for 'Pull Request' and their average turns

	Prompt	**Avg Turns**
I	From now on act as...	1
II	Provide a direct code	2
IV	Give code and ask to restructure	2.33
V	Give code and ask to diagnose the problem	5.5
VI	General question before task	1

categories. Table 4 displays the average number of turns for each of the prompt categories. For the more populated categories, the average number of turns is fairly even. The remaining categories, while they do have lower averages, only include 1 or 2 instances in each category, making it difficult to draw conclusions. These instances could be outliers or unrepresentative of the average turns if there was more data. However, the prompt categories that included code all had lower averages than those without. The prompts with the highest averages were (VI) 'General question before task' and (VII) 'Generate code and refactor'. This may be due to the higher complexity and the large volume of content concerned with these tasks. We observed that some conversations beginning with a more general

question included turns spent by the developer trying to better understand the content. The 'File' category additionally had 2 unmerged, or 'No' instances. These two started with the 'Provide direct code' and 'Give code and ask to restructure' prompts, but they seem to be outliers in this sample section. The direct code instance provided extremely lengthy code and documentation, taking up multiple turns at the beginning. This suggests that being slightly specific can be more successful that giving ChatGPT every piece of information about the project. For the second unmerged instance, the programmer believes that ChatGPT failed due to the developer's own lack on knowledge in TypeScript. This suggests that ChatGPT should be used only in addition to the developer's own code, and the developer still needs to understand and work with the content produced by ChatGPT. This section also contained a fairly large percentage of 'Supplementary Info' conversations, totaling 19 instances out of 56 unique instances in the 'File' category. In the majority of the instances, the content produced from the conversations was definitely used in the project, which was typically a website, README, or article. The rest of the instances contained conversations that did not necessarily produce concrete content, but it was reasonable to assume that the developer used the conversation as supplementary information. Even though ChatGPT did not directly refactor code in these instances, it is important to note that ChatGPT was still successful in helping the developer in the realm of refactoring. In these situations, developers used similar prompt patterns, such as giving ChatGPT the role of an expert.

4.2.3. Issues

Table 5 displays the prompts used in the 'Issue' category. Since only 2 instances were merged to the final code, the only 2 successful prompts displayed were 'Give code and ask to restructure' and 'General question before task.' Both of these instances only resulted in one response from ChatGPT. Because the number of merged instances in this category is small, it would be inaccurate to draw conclusions about the usage of these prompts when describing an issue in a GitHub repository. When examining the instances that were not merged, however, one of the most common prompts used are Prompt I (From now on act as...). This would suggest that, while it does not seem to achieve desired results, developers will likely prompt ChatGPT to act as a software specializing in their specific issue before going on the describe the problem. A possible reason for the lack of merged results could be ChatGPT misunderstanding this prompt in certain contexts and failing to refactor the code while role playing as the described software.

4.2.4. Pull Request

Table 6 shows the prompts in the Pull Request category. The leading prompt within the 'Pull Request' category was 'Give code and ask to restructure,' with the next most frequent categories also showing that users of ChatGPT prefer to paste the code directly into the program and ask for reassessment. The prompt with the highest turn average was 'Give code and ask to diagnose the problem,' suggesting that ChatGPT may take longer to give the desired response

when tasked with finding errors in sections of code with missing context. While many of the instances within the 'Pull Request' category were merged to the final repository, other instances that included code that were not merged often started with Prompt VI (General question before task). The lack of usage of the code resulting from these conversations could suggest that developers utilizing ChatGPT similar to a search engine when trying to resolve a general issue within their code will not lead to their desired results, especially if the code they are working on is not provided to ChatGPT.

RQ2 Summary. Despite the lack of unique instances in certain categories, the trends in average turns and merge status across categories types allow us to draw conclusions concerning what type of prompt is most effective in certain environments. In the 'Commit' instances, we see that 'From now on...' prompts and direct code snippets were extremely efficient compared to prompts asking ChatGPT to diagnose the problem. The 'Pull Request' category also realized 'Give code and ask to diagnose the problem' as having the highest average length, and most 'no' instances starting with a general question. 'File' instances starting with a general question or request to generate code content had the highest average number of turns. Overall, the prompts requesting code review, generation, or explanation are generally less efficient when it comes to length of conversations. It also seems marginally more effective to be specific in terms of ChatGPT's role (e.g., expert, contributor) task, and code material being worked with.

4.3. RQ3: With which programming language does ChatGPT give the most satisfactory responses about refactoring?

Motivation. After all the data had been compiled and distributed evenly among the four categories, Commit, File, Pull Request, and Issue, we had observed that the dataset contained 23 coding languages, which we deemed to be very diverse considering a dataset of 252 unique instances. After observing the kinds of questions developers would ask ChatGPT in various circumstances through the development process, we asked if the language ChatGPT was responding to would be a sensible variable to measure within the study to help developers understand how to further utilize this tool in their development process. For example, if developers were more likely to ask ChatGPT for advice with procedural programming languages, it would be a practical conclusion of this study to recommend developers to seek the aid of ChatGPT when working with languages like C or Pascal rather than scripting languages.

Approach. Instead of only considering instances where ChatGPT-generated code was merged with the final program, we considered the total number of unique instances to account for all scenarios where developers sought the help of ChatGPT. We then organized the instances by their language, and made sure to mark down whether the instance was 'Exact', 'Modified', or was not merged into the final code at all, those of which were marked 'No'. After the data had

been compiled, we display the distribution of languages in each category, along with the number of 'Exact', 'Modified', and 'No' instances for each language. We then drew conclusions based on the resulting data.

Results. We determined the languages that resulted in the most successful feedback from ChatGPT while collaborating on refactoring assignments. Similar to our earlier examinations, redundant exchanges were sifted through so that every distinct conversation or prompt was not duplicated in the count.

4.3.1. Commits

Figure 4 showcases the different languages that developers use in their commits and demonstrates Cascading Style Sheets (CSS) as a standout choice among others. In the case of CSS, there were 138 unique occurrences where developers adopted ChatGPT's recommendations into their final commit, out of which 120 were exact matches while 2 had slight modifications and 3 were not adopted. The high number of instances with CSS implies that developers perceived ChatGPT's guidance as highly effective and precise when dealing with tasks related to CSS refactoring. For other languages, the chart shows minimal usage, with Hypertext Markup Language (HTML) having a total of 7 instances (all exact matches), Shell with one instance being modified the other was not adopted, and Dockerfile with 1 instance (exact match). The 'Commit' category heavily embraces CSS because it has a direct bearing on the visual presentation and user interface— aspects that see frequent updates with finer refinements. The small nature of CSS changes makes them more incremental; hence, ideal for constant commits. An automation of CSS tasks would simplify these updates; guaranteeing uniform styling and faster iterations. Furthermore, given that changes to CSS are typically less risky as compared to modifications in back end code, they allow for more frequent commits.

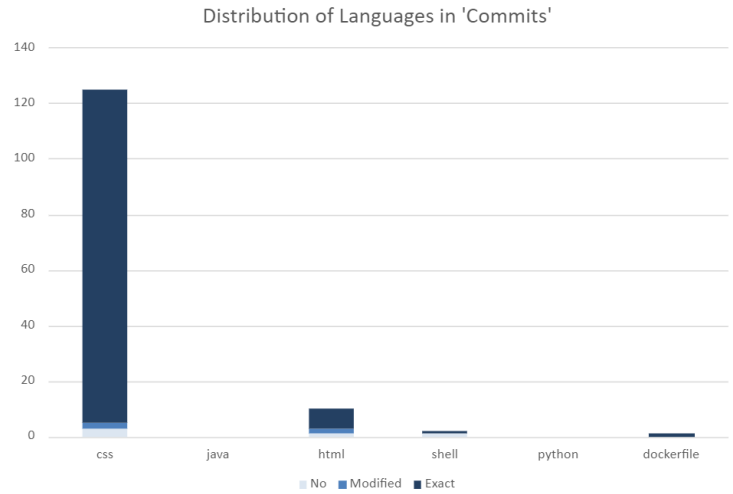


Figure 4: The number of unique instances for each language in the 'Commit' category

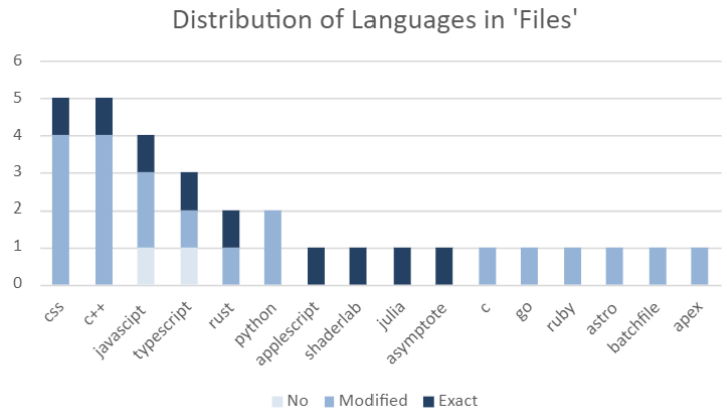


Figure 5: The number of unique instances for each language in the 'File' category

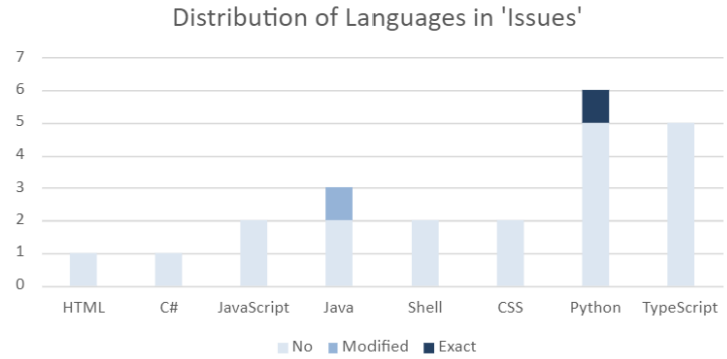


Figure 6: The number of unique instances for each language in the 'Issues' category

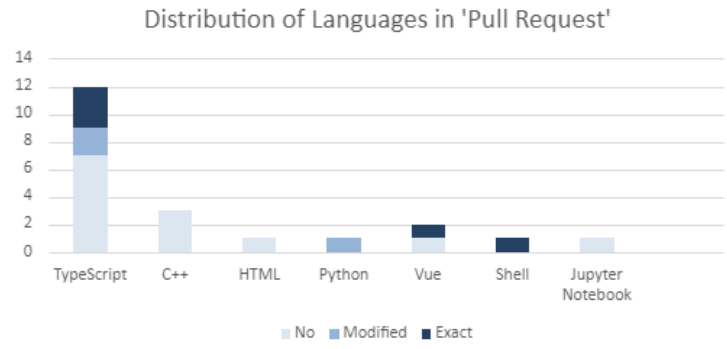


Figure 7: The number of unique instances for each language in the 'Pull Request' category

4.3.2. Files

Figure 5 displays the wide variety of languages featured in the Files category, with the majority of Files being in CSS, C++, JavaScript, and TypeScript, contrasting the sparse distribution throughout the other languages. CSS and C++ led in popularity with 5 instances each, both having a 1:4 exact to modified ratio. As thoroughly explained in the 'Commit' (Section 4.3.1), CSS is heavily involved in front-end web development. The projects in 'File' are often web pages or applications, which explains the high frequency of CSS, JavaScript, and TypeScript projects. JavaScript and TypeScript had 4 and 3 instances respectively, each with 1 exactly-merged instance and 1 unmerged 'No' instance. All of the instances in the rest of the language categories were merged to the repository. This means that practically all ChatGPT conversations producing code were used, either exactly or with some modifications. The fact that the only two unmerged instances were in JavaScript and TypeScript, a superset of JavaScript, could indicate that ChatGPT is not as effective in helping with refactoring tasks with these languages due to the extensive documentation and nuanced nature of JavaScript and larger-scale applications done with TypeScript.

4.3.3. Issues

Figure 6 displays the wide variety of languages used within the 'Issue' category, showing the continued interest in utilizing ChatGPT as a refactoring tool from developers when it comes to issues in their GitHub repository. However, only two of these instances contained code suggestions from ChatGPT that were pushed to the final version, one in Java and one in Python. While the number of instances in this category is limited, there is a preference for languages that support object-oriented programming, namely TypeScript and Python. This could lead to the presumption that developers are more likely to turn to ChatGPT when faced with an instance in the 'Issue' category that deals with object-oriented programming. Creating an 'Issue' in GitHub is a common procedure for developers throughout the design process, particularly lengthy ones, so that they can keep track of parts of the code that they plan to work on at a later time. Thus, a possible reason for the abundance of instances that focused on a language that supports object-oriented programming could be that 'Issue' instances in GitHub are more likely to be seen in repositories that plan to contain an extensive amount of code. This could lead to a specific problem in the repository that developers may not have met before, causing them to turn to a tool like ChatGPT for advice.

4.3.4. Pull Request

Figure 7 displays the variety of languages featured in the 'Pull Request' category, clearly showing a favor for TypeScript among the other selected languages. The data for TypeScript shows 8 unique programmers who utilized ChatGPT in their TypeScript project, 3 of whom used the suggestions given by ChatGPT in the final version of their code. The category of 'Pull Request' involves developers suggesting changes to an existing repository, most of the time to solve existing errors or bugs in the code. A possible reason why TypeScript is the favorite in this category among the many other languages is that it is used

to develop large-scale applications, a process that often leads to oversights or seemingly inexplicable error messages. Therefore, it would make sense for developers to suggest ChatGPT refactoring through pull requests, as TypeScript is a commonly used collaborative language where developers will likely turn to outside resources such as ChatGPT for aid.

RQ3 Summary. Across all 4 categories, the most common language that developers ask ChatGPT for refactoring aid with, and the language with the highest number of merged instances, is CSS. The 'Commit' category show 138 unique CSS instances, 120 of which were merged exactly. The high number of exact merges is likely due to CSS changes within a repository being incremental and less risky than back end changes, which are also easier for ChatGPT to aid. The 'File' category's leading language was CSS as well, and every instance except 2 contained a code snippet that was merged to the final code across all languages, either modified or exact, leading to the assumption that developers coding in most languages find ChatGPT helpful as a refactoring tool when it comes to editing whole files. The 'Issue' category seems to have the opposite trend however, with only 2 instances being merged across all languages, mostly in Python and TypeScript, indicating a dissatisfaction from developers involving ChatGPTs refactoring ability of errors or bugs. 'Pull Request' suggests a similar conclusion that developers are unsatisfied with ChatGPT's refactoring while building large-scale projects with dynamic languages.

5. Discussion

Takeaway #1: Developers found Chat Generative Pre-trained Transformer (ChatGPT) to be best at refactoring 'Commit' instances. According to RQ1, while observing amount of instances marked 'Exact' in each category, it is clear that developers were able to use the exact code generated by ChatGPT more frequently when refactoring 'Commit' instances than any other category. The 'Commit' category contains 144 total instances of developers using any code given in their conversations with ChatGPT, 140 (97.2%) were marked 'Exact', higher than any other category measured in this study. This insinuates that ChatGPT is employed best for dealing with short, small changes to the code, which are categorized mostly into 'Commit' instances. When refactoring code, ChatGPT may not be useful to developers for long or complex data structures, but when refactoring smaller or less risky aspects, such as the front end of the code, developers seem to be satisfied with the exact code given by ChatGPT. It might be worth exploring whether that matches the academic definition of refactoring Fowler et al. (1999); Mens & Tourwé (2004).

Takeaway #2: It is best to prompt ChatGPT to "act as" a separate software that aims to specifically code what the developer needs. After taking note of the several kinds of prompts developers chose to lead with

in the Developer-Generative Pre-trained Transformer (DevGPT) dataset while answering RQ2, the most prevalent one is a prompt that asks ChatGPT to behave as a separate Artificial Intelligence (AI) assistance tool that aims specifically to aid programmers in their development process. This is seen in Table 3, displaying the multiple prompts in the 'Commit' category, with the aforementioned prompt holding 57 unique instances within the dataset, delivering a code snippet after an average of 1.58 turns from the developer. Although other categories did not see as large an impact from this prompt, we believe that telling ChatGPT to "act as" a separate AI programming assistant will likely lead to straightforward responses in resulting code. At the very least, this method may serve as an alternative to developers who have tried to utilize ChatGPT as a refactoring tool and are unsatisfied with its response when being direct.

Takeaway #3: ChatGPT is sought to be used most by developers working on large-scale applications or websites. Figures 4, 5, 6 and 7 display the distribution of languages within each category within the DevGPT dataset, the two most popular overall being CSS and TypeScript. As specified by RQ3, after observing individual instances within each category further, it seems that most developers who go to ChatGPT for help require aid for development in either one aspect of a website using CSS, as shown best in the 'Commit' instances, or a web application using TypeScript, utilized most within the 'Pull Request' category. While there may be an LLM in the future that could handle refactoring extensive databases, developers would be wise to limit the use of the current version of ChatGPT to refactoring front end issues, shown by the extensive number of merged CSS 'Commit' instances.

Takeaway #4: The difference in experiment results between 'File' and 'Commit' instances can be attributed to complexity and amount of material being covered. According to our results in RQ1, 97% of merged 'Commit' instances were utilized without modifications, as opposed to only 31% of the merged 'File' instances. Furthermore, as clearly depicted in Figure 3, there is a drastic difference between the two categories when looking at RQ2, which analyzed the average number of turns. This noticeable disparity was unexpected, as the only major difference between 'File' and 'Commit' instances is the form of content the ChatGPT conversation is linked to. The 'File' instances are linked to whole projects or repositories, as opposed to a single 'Commit'. This provides a reason for the variation in RQ1; 'Commit' instances are labeled as such because the content was definitely utilized, whereas conversations linked to files were not necessarily fully committed to the material given. The outstanding 'Files' results for RQ2 might have a similar root explanation. When users label instances to a code file, we noticed it often indicated that the ChatGPT conversation concerns multiple tasks for the project, or that the projects and tasks were more complex, increasing the turns spent on the topic and the complications that followed. These discrepancies are crucial to point out because they would otherwise be attributed to the variables we are testing, such as the type of prompt or programming language. It also aids in understanding how developers are interacting with ChatGPT and GitHub in the refactoring process.

Takeaway #5: ChatGPT exceeded expectations with respect to its abilities to refactor in niche, less popular programming languages. As shown in Figure 5, the niche language categories with only one file, such as ShaderLab, Asymptote, and AppleScript, contained the ChatGPT code exactly as provided, indicating ChatGPT was successful despite the possible deficiency of information on these languages in comparison to widely-used languages like Python and C++. Furthermore, the cases written in new subset languages of Rust defined by the user were also successful, one being specifically merged and the other with some modifications. This indicates that ChatGPT is able to be a successful refactoring tool when working in a variety of languages, and it is not necessarily limited to popularly used languages.

Takeaway #6: Although ChatGPT is a useful tool for some coding tasks, more development may be necessary to make it suitable for more complicated problem-solving situations. Figure 2 shows that developers can benefit from ChatGPT generated code while refactoring, especially when integrating code after 'Commit'. The tool's ability to make the refactoring process is demonstrated by the fact that developers frequently find the generated code usable, either exactly as it is or with a few minor modifications. Its efficacy is less evident when attempting to resolve particular problems inside repositories. To save time and effort, developers should use ChatGPT for easier refactoring tasks. However, they should not rely solely on ChatGPT for complex problems, as it may not provide reliable solutions, necessitating human oversight to ensure code dependability and quality.

Takeaway #7: A balance between automation and human oversight is necessary for the effective use of ChatGPT for refactoring. Table 3 shows that prompts such as "From now on act as..." and direct code provisions lead to satisfactory results with fewer interactions, indicating effective initial automation steps. Although ChatGPT's quick and frequently accurate code suggestions can greatly speed up the refactoring process, developers must carefully analyze and validate these suggestions. Although simple and repetitive refactoring activities can be easily handled by automation, human inspection is still required to ensure the accuracy, functionality, and security of the code. To protect the quality of the program, developers who would use AI to incorporate their code should incorporate ChatGPT into their workflows as a helpful tool that boosts output while adhering to strict code review procedures.

6. Threats To Validity

Lack of Unique Instances. The Developer-Generative Pre-trained Transformer (DevGPT) dataset was selected based on its numerous Chat Generative Pre-trained Transformer (ChatGPT) conversations with thousands of code snippets contained across multiple categories. However, when parsing through the dataset to exclude all instances that do not involve refactoring, the number of unique ChatGPT conversations becomes limited. Although DevGPT includes six categories of conversations, two of them, 'Discussion' and 'Hacker News',

had to be excluded from this study due to a lack of conversations. Some categories included in the study also suffer from a lack of instances, although not enough to be excluded, with 'Issue' only having 2 instances of conversations that included refactored code that were merged to the final repository. These concerns are addressed by the inclusion of multiple other categories containing an abundance of unique instances that could be reasonably analyzed to discuss the practicality of ChatGPT, notably the 'Commit' category.

Dataset Coverage and Generalizability. The DevGPT dataset forms the basis of our analysis because it contains developer–ChatGPT dialogues voluntarily shared through GitHub projects. As a result, it reflects a self-selected group of developers, and the associated projects may differ in size, complexity, and domain from other use cases. The findings should be viewed as evidence of patterns within this particular dataset rather than as universally generalizable trends. Furthermore, because the dataset comprises developers who choose to publicly share their conversations via tools such as ChatGPT, self-selection may limit the generalizability of the findings to private, enterprise-level development environments. Future studies could mitigate this limitation by validating the results with additional datasets, including those from closed-source or proprietary codebases. Moreover, the key tables and figures in this paper (for example, Table 1, Table 2, Figure 2a, and Figure 3) illustrate how we filtered, categorized, and analyzed the data, and make explicit which subsets support each research question.This clarification enables readers to evaluate the extent to which our conclusions can be applied to other development environments and to identify areas requiring further validation.

Subjective Categorization. For RQ2, the prompts were designed by the authors after manually examining each instance across the dataset and taking note of common practices in between the developer's messages. However, when categorizing the instances into the final seven prompts decided upon, the process was manual, as we decided to look at the first message sent by the user and decide which prompt category the conversation fell into. This opens up the possibility of a difference in opinion from the authors affecting the results, where the same conversation could have been filtered into different prompt categories. To combat this circumstance, the authors discussed the prompt categorization regularly, and agreed on keywords and phrases that would automatically qualify an instance for a specific category. Any instance still unable to be categorized were highlighted and discussed by all authors until a prompt category was agreed upon for that instance.

7. Conclusion

In this study, we investigated Chat Generative Pre-trained Transformer (ChatGPT's) potential as a tool to help developers with code reworking. Our empirical investigation, which made use of the Developer-Generative Pre-trained Transformer (DevGPT) dataset, provided information about the circumstances in which developers can use ChatGPT most efficiently. By manually analyzing and categorizing prompts related to refactoring, we were able to find how

developers can best utilize ChatGPT for a wide range of refactoring activities and improve their code quality. According to the study, ChatGPT can help developers with a range of refactoring activities, showing that it can improve code quality. However, the prompt's precision and clarity significantly impacted ChatGPT's effectiveness. It is important to emphasize that developers who emphasize rapid formulation receive more accurate and helpful responses from users when they provide clear instructions and a rich context. For RQ1, how helpful is the code generated by ChatGPT in assisting developers with refactoring tasks, according to research, ChatGPT is a helpful resource for developers working on refactoring projects because it offers helpful and efficient code recommendations. A large percentage of the code produced by ChatGPT was either combined exactly or with only slight changes, indicating its usefulness in practical applications. The tool was particularly good at offering simple code enhancements and changes, which developers could easily include in their projects. However, developers had to make extra modifications for more complicated refactoring activities, indicating that although ChatGPT is useful, it works best when used as a supplement rather than a stand-alone solution.

For RQ2, what is the best way to prompt ChatGPT to get a satisfactory response about refactoring with the fewest possible interactions, according to the research, the most successful prompts are those that give ChatGPT precise instructions right away. Developers received more accurate and helpful responses with fewer engagements when they included code snippets, background information, and explicit task requirements in their initial prompts. Particularly successful prompts that requested ChatGPT to "act as" a different software program made expressly to help with coding produced faster and more satisfactory results.

For RQ3, with which programming language does ChatGPT give the most satisfactory responses about refactoring, It was discovered that ChatGPT was most successful in providing good refactoring solutions for CSS. Developers used ChatGPT a lot for refactoring jobs involving CSS, and a high percentage of exact merges indicated meaningful and accurate advice. On the other hand, ChatGPT's performance varied for less popular and more difficult languages, and the resulting code was clearly altered. This shows that ChatGPT needs additional fine-tuning to support a wider range of programming languages more successfully, even while it works incredibly well for some.

In summary, ChatGPT exhibits potential as a useful tool for developers, especially for work involving well-defined refactoring. To optimize the advantages of using ChatGPT in software development workflows, future research should focus on improving prompt techniques and investigating language-specific performance.

8. Future Work

Our research attempts to explore to what extent Chat Generative Pre-trained Transformer (ChatGPT) can currently aid developers in code refactoring, and furthermore, what conditions induce ChatGPT to do so most efficiently,

specifically considering two factors, prompt and language. The most immediate direction for future work is to incorporate real-time validation by testing our methodology directly through live developer workflows. This would allow us to confirm whether the trends we observed with the retrospective DevGPT dataset are corroborated in controlled development contexts. For example, we plan to integrate real-time validation via integrated development environment (IDE) plugins to assist with improving code quality. Ideally, these are proactive quality monitors that continuously evaluate refactoring impact. The main challenge is latency, as real-time validation must be nearly instantaneous to avoid interrupting the developer's workflow. Moreover, managing context windows is complex; the system must grasp the entire project structure to determine if a generated clone is unnecessary or helpful. There are many additional problems, including privacy and security issues when developers want to share their code context, the potential for solutions that could generate random results or hallucinations, and refactoring while preserving system behavior, even though there is no complete test coverage. Next, our prompt taxonomy and study procedure provide a strong foundation for exploring what type of prompt yields more efficient and successful results. However, as mentioned in Section 7, the dataset size limited our ability to analyze finer-grained prompt categories. A larger dataset would allow us to revisit these categories, for instance, revisit "Provide direct code" prompts but compare it to "Provide direct code, then present guidelines and tasks". Using a broader dataset in future work would also be crucial with our study of programming language variation. While our study reports a very high success rate with CSS, results do vary across other languages depending on category types ('Commits,' 'Files,' 'Issues,' and 'Pull Requests'). This change could crucially provide more reliable conclusions for languages that have fewer instances, ultimately helping developers determine which resources are most effective for specific languages. While our analysis centered on prompts, conversation length, and programming language, future research could analyze sentiment and token distribution patterns to enhance understanding of developer-LLM communication methods. The evaluation of token distribution and sentiment in developing LLM communication will help researchers understand how developer intent and conversational tone affect the quality of ChatGPT's refactoring suggestions. Finally, while our study focused on ChatGPT in isolation rather than integrated frameworks, e.g. Black Box, which targets code generation, future work could extend this research by comparing ChatGPT's refactoring effectiveness against such tools. This would provide a broader view of how standalone LLM use contrasts with specialized frameworks, and help developers choose which resources are most effective for their tasks. Beyond frameworks, additional condition factors, such as type of project or reason for refactoring, could be analyzed to further refine our knowledge of the areas ChatGPT is most effective and where it can be improved. Ultimately, deepening this line of research is critical to shaping best practices for LLM-assisted development, ensuring that these tools can reliably support improvements in code quality and maintainability.

While promising prompting techniques exist, exploring new, efficient methods in the growing field of LLMs is crucial. Agentic AI is another useful tool

that, with recent refinements and capabilities such as memory, search, cloud, and database access via Model Context Protocols, enables LLMs to self-fix, compile, check, and run code. This facilitates multi-step thinking, reduces errors, and avoids hallucinations. Multi-agent systems, in which multiple LLMs communicate, verify, and compare work, can perform roles across the software development lifecycle. By tracking updates made by these systems across multiple GitHub repositories, we can gain insight into the quality of these changes as both LLMs and the data they utilize evolve. This will expand the dataset, enabling more comprehensive conclusions and validating it across other contexts. Furthermore, current tools frequently lack the "project memory' needed to recognize when they are introducing quality issues such as repeating logic. Our research into developer interactions and model performance indicates that future tools should incorporate "Project-Level RAG" (Retrieval-Augmented Generation) to include examples of best practices when refactoring the code, while also drawing from the repository's architectural patterns, prior commits, and established abstractions. Again, by anchoring AI suggestions to the patterns in the existing codebase, future systems can proactively detect and prevent the occurrence of "accidental clones". But this nuanced approach will enable AI assistants to do more than just generate usable code; they'll also help design systems that align with established project standards. **Declaration of generative AI and AI-assisted technologies in the writing process.** During the preparation of this work, the authors used the ChatGPT Web interface to improve the language and readability. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

(). `https://sites.google.com/stevens.edu/refactoringprogramspinnacle24/`.

AlOmar, E. A., Venkatakrishnan, A., Mkaouer, M. W., Newman, C. D., & Ouni, A. (2024). How to refactor this code? an exploratory study on developer-chatgpt refactoring conversations. In *2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)* (pp. 202–206). IEEE.

AlOmar, E. A., Xu, L., Martinez, S., Peruma, A., Mkaouer, M. W., Newman, C. D., & Ouni, A. (2025). Chatgpt for code refactoring: Analyzing topics, interaction, and effective prompts. IEEE.

Biswas, S. (2023). Role of chatgpt in computer programming.: Chatgpt in computer programming. *Mesopotamian Journal of Computer Science*, *2023*, 8–16.

Chavan, O. S., Hinge, D. D., Deo, S. S., Wang, Y., & Mkaouer, M. W. (2024). Analyzing developer-chatgpt conversations for software refactoring: An exploratory study. In *Proceedings of the 21st International Conference on Mining Software Repositories* (pp. 207–211).

Chouchen, M., Bessghaier, N., Begoug, M., Ouni, A., Alomar, E., & Mkaouer, M. W. (2024). How do software developers use chatgpt? an exploratory study on github pull requests. In *Proceedings of the 21st International Conference on Mining Software Repositories* (pp. 212–216).

Das, J. K., Mondal, S., & Roy, C. (2024). Investigating the utility of chatgpt in the issue tracking system: An exploratory study. In *Proceedings of the 21st International Conference on Mining Software Repositories* (pp. 217–221).

De Martino, V., Martínez-Fernández, S., & Palomba, F. (2024). Do developers adopt green architectural tactics for ml-enabled systems? a mining software repository study. *arXiv preprint arXiv:2410.06708*, .

DePalma, K., Miminoshvili, I., Henselder, C., Moss, K., & AlOmar, E. A. (2024). Exploring chatgpt's code refactoring capabilities: An empirical study. *Expert Systems with Applications*, *249*, 123602.

Dilhara, M., Bellur, A., Bryksin, T., & Dig, D. (2024). Unprecedented code change automation: The fusion of llms and transformation by example. *arXiv preprint arXiv:2402.07138*, .

Divyansh, S., Apoorva, P., Singh, S. S., Ershadi, A., Makwana, H., & AlOmar, E. A. (2025). Evaluating the effectiveness of chatgpt in improving code quality. In *2025 IEEE 4th International Conference on Computing and Machine Intelligence (ICMI)* (pp. 1–7). IEEE.

Fowler, M., Beck, K., Brant, J., Opdyke, W., & Roberts, d. (1999). *Refactoring: Improving the Design of Existing Code*. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc. URL: `http://dl.acm.org/citation.cfm?id=311424`.

Hao, H., Hasan, K. A., Qin, H., Macedo, M., Tian, Y., Ding, S. H., & Hassan, A. E. (2024). An empirical study on developers shared conversations with chatgpt in github pull requests and issues. *arXiv preprint arXiv:2403.10468*, .

Haque, M. A., & Li, S. (2023). The potential use of chatgpt for debugging and bug fixing. *EAI Endorsed Transactions on AI and Robotics*, *2*, e4–e4.

Jin, K., Wang, C.-Y., Pham, H. V., & Hemmati, H. (2024). Can chatgpt support developers? an empirical evaluation of large language models for code generation. In *2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)* (pp. 167–171). IEEE.

Khoury, R., Avila, A. R., Brunelle, J., & Camara, B. M. (2023). How secure is code generated by chatgpt? *arXiv preprint arXiv:2304.09655*, .

Liu, B., Jiang, Y., Zhang, Y., Niu, N., Li, G., & Liu, H. (2024). An empirical study on the potential of llms in automated software refactoring. *arXiv preprint arXiv:2411.04444*, .

Liu, J., Xia, C. S., Wang, Y., & Zhang, L. (2023). Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *arXiv preprint arXiv:2305.01210*, .

Ma, W., Liu, S., Wang, W., Hu, Q., Liu, Y., Zhang, C., Nie, L., & Liu, Y. (2023). The scope of chatgpt in software engineering: A thorough investigation. *arXiv preprint arXiv:2305.12138*, .

Martinez, S., Xu, L., Elnaggar, M., & Alomar, E. A. (2025). Software refactoring research with large language models: A systematic literature review. *Journal of Systems and Software*, (p. 112762).

Mens, T., & Tourwé, T. (2004). A survey of software refactoring. *IEEE Transactions on software engineering*, *30*, 126–139.

Mondal, S., Bappon, S. D., & Roy, C. K. (2024). Enhancing user interaction in chatgpt: Characterizing and consolidating multiple prompts for issue resolution. In *Proceedings of the 21st International Conference on Mining Software Repositories* (pp. 222–226).

Pomian, D., Bellur, A., Dilhara, M., Kurbatova, Z., Bogomolov, E., Bryksin, T., & Dig, D. (2024a). Together we go further: Llms and ide static analysis for extract method refactoring. *arXiv preprint arXiv:2401.15298*, .

Pomian, D., Bellur, A., Dilhara, M., Kurbatova, Z., Bogomolov, E., Sokolov, A., Bryksin, T., & Dig, D. (2024b). Em-assist: Safe automated extractmethod refactoring with llms. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering* (pp. 582–586).

Shirafuji, A., Oda, Y., Suzuki, J., Morishita, M., & Watanobe, Y. (2023). Refactoring programs using large language models with few-shot examples. In *2023 30th Asia-Pacific Software Engineering Conference (APSEC)* (pp. 151–160). IEEE.

Sobania, D., Briesch, M., Hanna, C., & Petke, J. (2023). An analysis of the automatic bug fixing performance of chatgpt. *arXiv preprint arXiv:2301.08653*, .

Sun, W., Fang, C., You, Y., Miao, Y., Liu, Y., Li, Y., Deng, G., Huang, S., Chen, Y., Zhang, Q. et al. (2023). Automatic code summarization via chatgpt: How far are we? *arXiv preprint arXiv:2305.12865*, .

Tian, H., Lu, W., Li, T. O., Tang, X., Cheung, S.-C., Klein, J., & Bissyandé, T. F. (2023). Is chatgpt the ultimate programming assistant–how far is it? *arXiv preprint arXiv:2304.11938*, .

Tufano, R., Mastropaolo, A., Pepe, F., Dabić, O., Di Penta, M., & Bavota, G. (2024). Unveiling chatgpt's usage in open source projects: A mining-based study. In *2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)* (pp. 571–583). IEEE.

White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., & Schmidt, D. C. (2023). A prompt pattern catalog to enhance prompt engineering with chatgpt. *arXiv preprint arXiv:2302.11382*, .

White, J., Hays, S., Fu, Q., Spencer-Smith, J., & Schmidt, D. C. (2024). Chatgpt prompt patterns for improving code quality, refactoring, requirements elicitation, and software design. In *Generative AI for Effective Software Development* (pp. 71–108). Springer.

Xia, C. S., & Zhang, L. (2023). Keep the conversation going: Fixing 162 out of 337 bugs for 0.42 each using chatgpt. *arXiv preprint arXiv:2304.00385*, .

Xiao, T., Hata, H., Treude, C., & Matsumoto, K. (2024a). Generative ai for pull request descriptions: Adoption, impact, and developer interventions. *Proceedings of the ACM on Software Engineering*, *1*, 1043–1065.

Xiao, T., Treude, C., Hata, H., & Matsumoto, K. (2024b). Devgpt: Studying developer-chatgpt conversations. In *2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)* (pp. 227–230). IEEE.

Yokoi, K., Choi, E., Yoshida, N., Okada, J., & Higo, Y. (2023). Cost-benefit analysis for modernizing a large-scale industrial system. In *2023 30th Asia-Pacific Software Engineering Conference (APSEC)* (pp. 441–449). IEEE.

Yu, X., Liu, L., Hu, X., Keung, J. W., Liu, J., & Xia, X. (2024). Fight fire with fire: How much can we trust chatgpt on source code-related tasks? *arXiv preprint arXiv:2405.12641*, .